Device Panel Loading

Overview

When you select a device in T3000, the system loads all data from that device panel and displays it in your interface. This process involves network communication, database caching, and real-time updates.

What Happens When You Click a Device?

1. You Select a Device

Click on any device in the device tree (left sidebar). The device could be a T3-BB controller, T3-TB thermostat, or any BACnet device on your network.

2. Loading Indicator Appears

You'll see a loading spinner while the system retrieves data from the device.

3. Data Appears in Tabs

Once loaded, you'll see multiple tabs with device data:

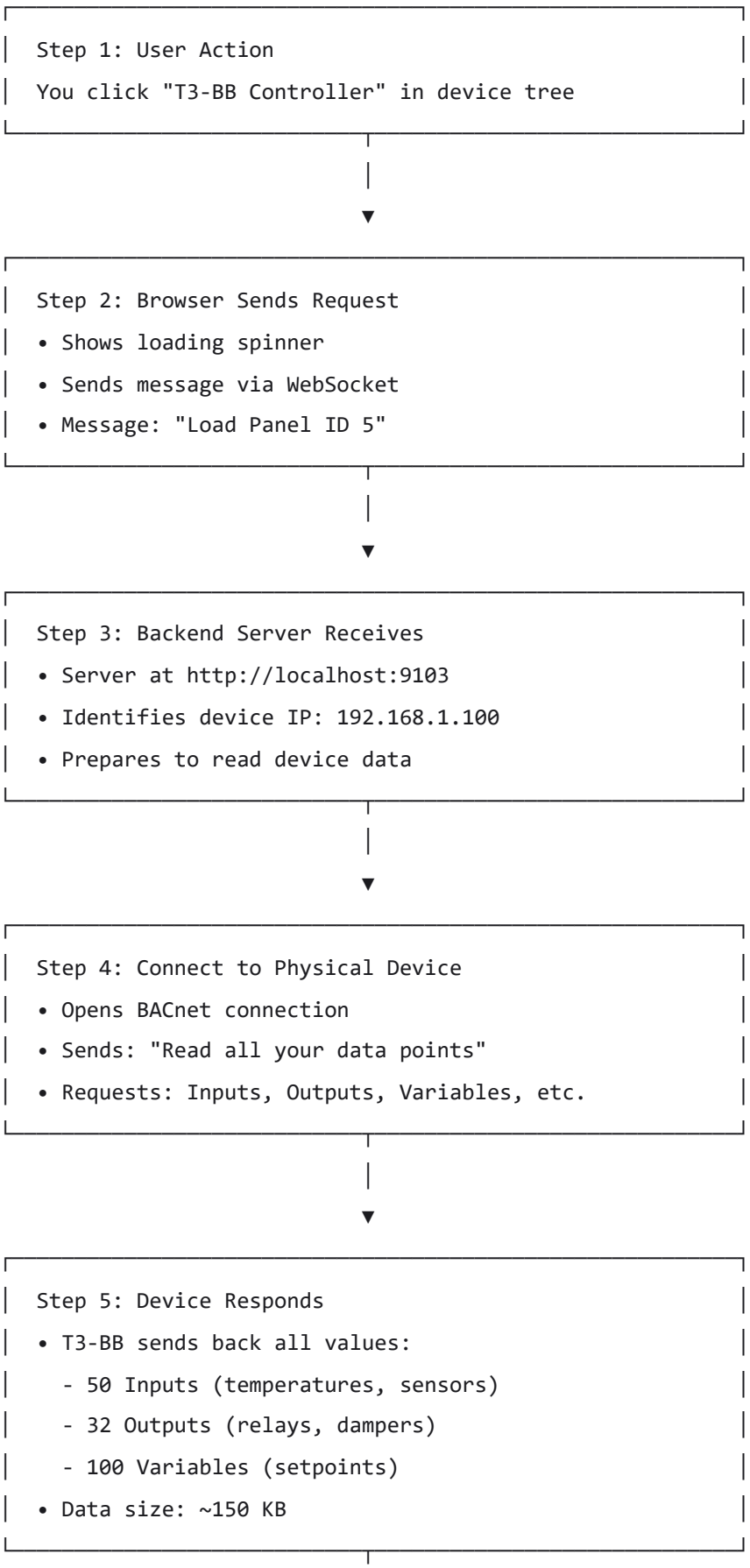
- **Inputs**: Sensor readings (temperature, humidity, pressure)
- **Outputs**: Control points (relays, dampers, valves)
- **Variables**: Setpoints and calculated values
- **Programs**: Control logic
- **Schedules**: Time-based automation
- **Graphics**: Visual layouts

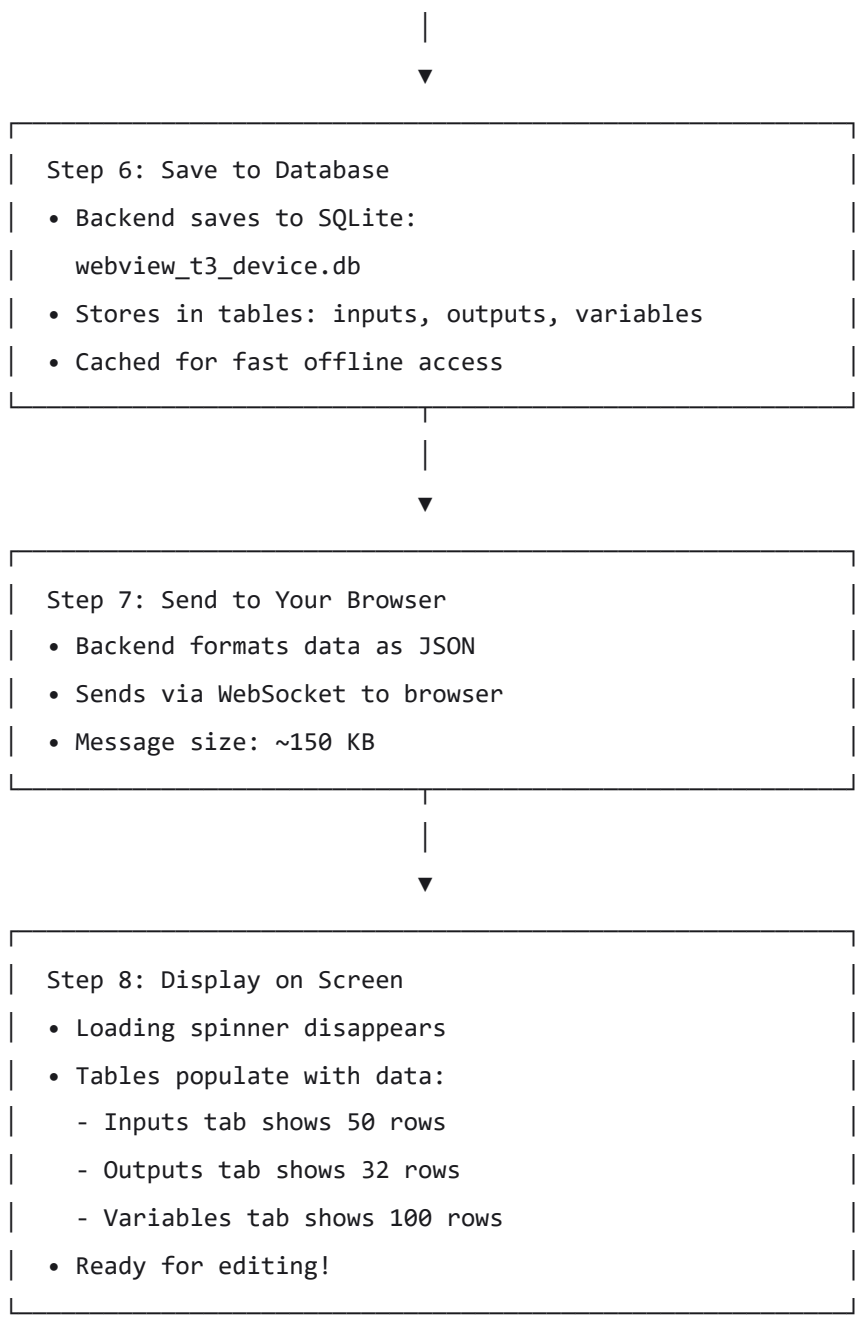
4. Real-Time Updates

After initial load, any changes on the device automatically update in your interface without refreshing.

Data Flow Visualization

TEXT





Total Time: 2-5 seconds for typical device

Why This Matters

Fast Offline Access: Once loaded, device data is cached locally. You can view it even if the device goes offline.

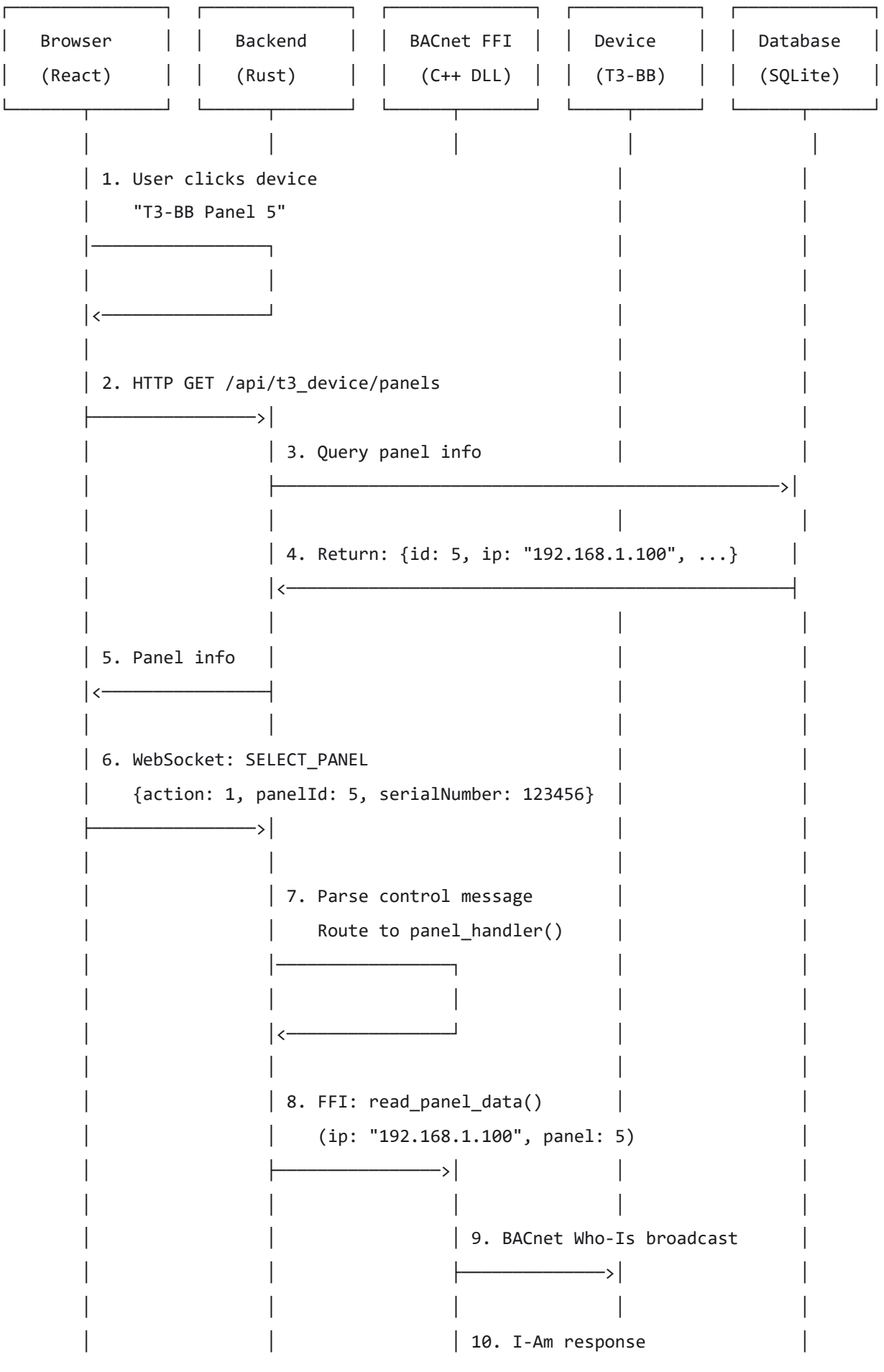
Real-Time Updates: Changes on the physical device appear in your interface automatically.

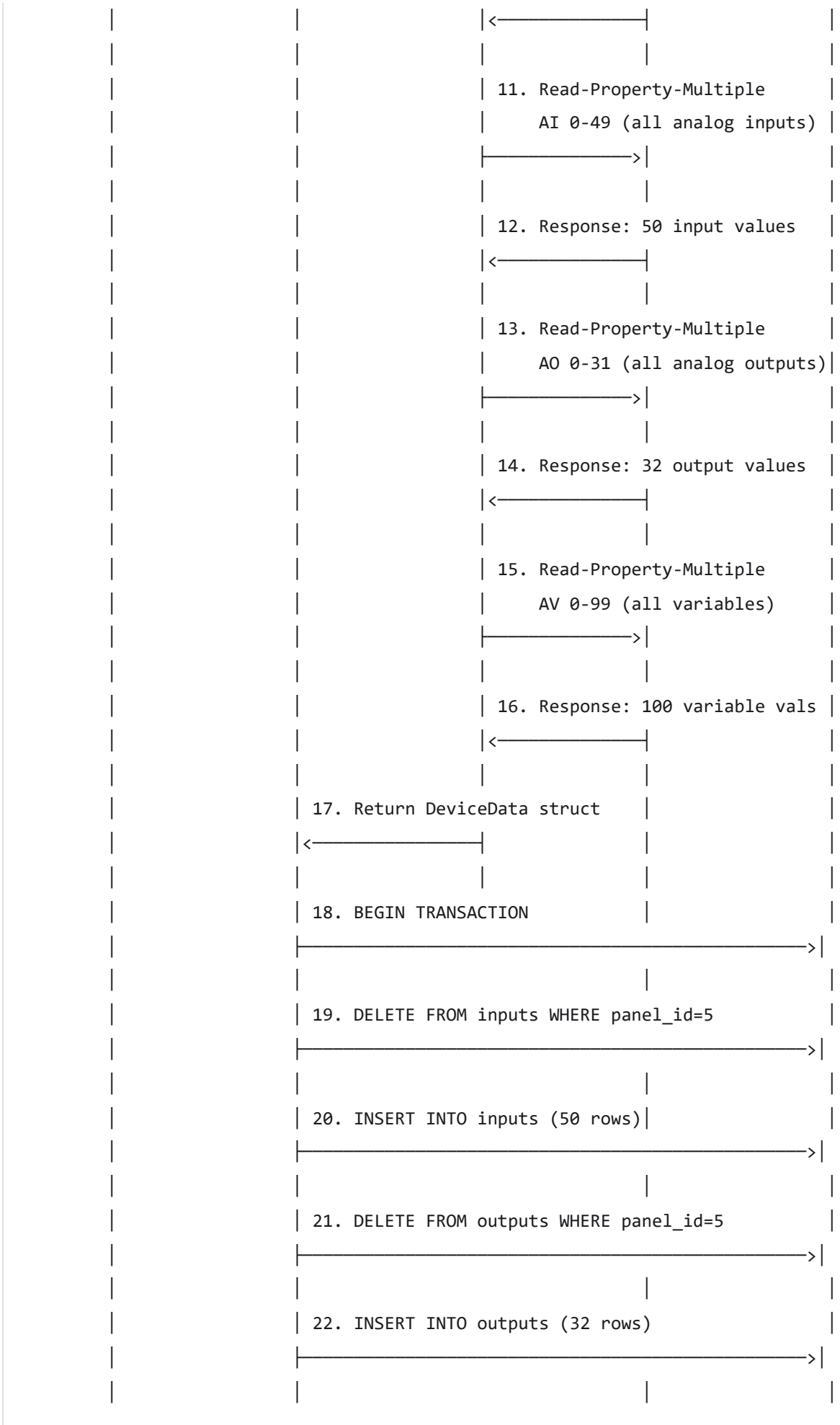
Efficient Network Usage: The system only loads data when you select a device, not all at once.

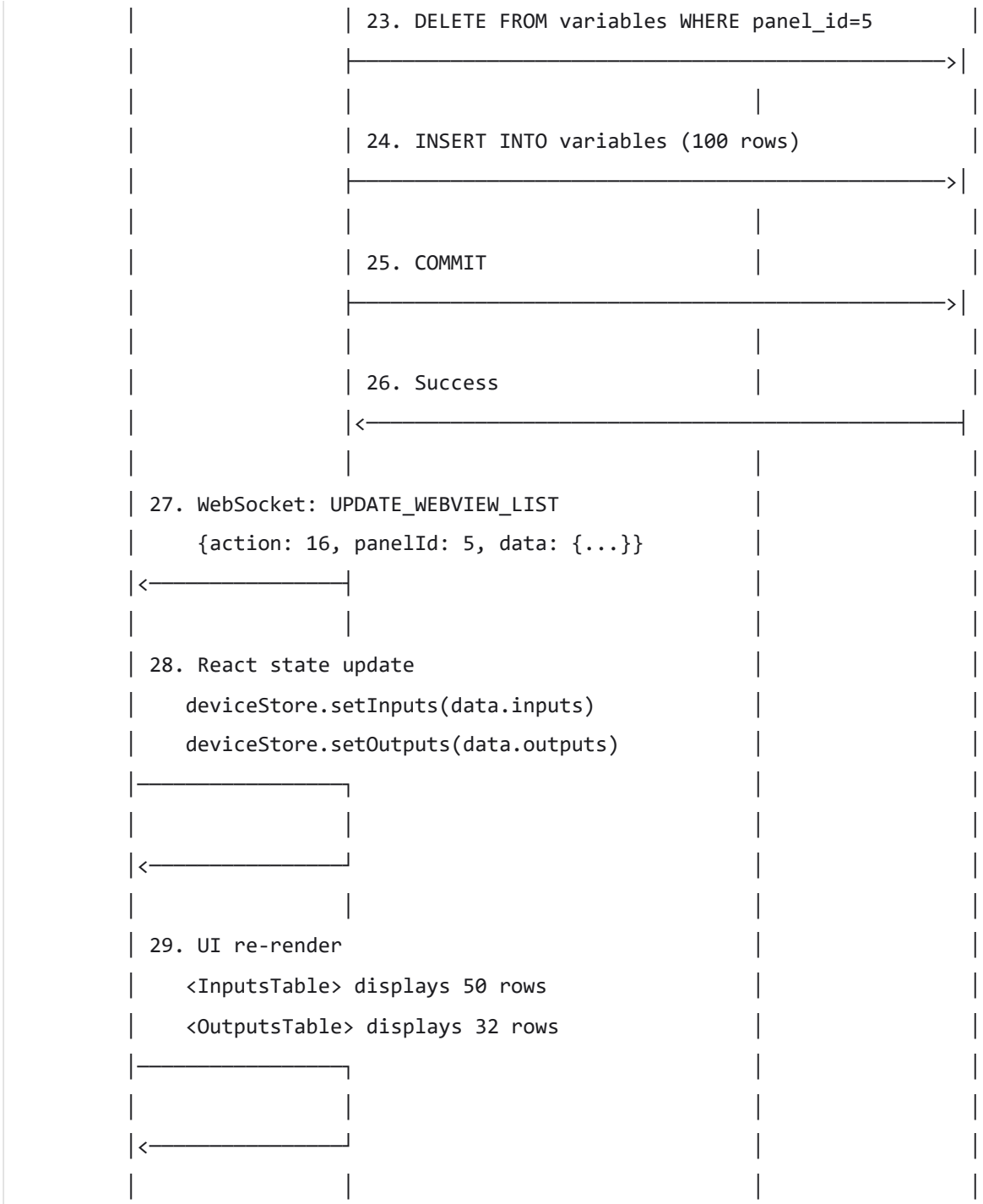
Complete Technical Data Flow

Full System Sequence Diagram

TEXT


Browser (React)
Backend (Rust)
BACnet FFI (C++ DLL)
Device (T3-BB)
Database (SQLite)
1. User clicks device "T3-BB Panel 5"
2. HTTP GET /api/t3_device/panels
3. Query panel info
4. Return: {id: 5, ip: "192.168.1.100", ...}
5. Panel info
6. WebSocket: SELECT_PANEL {action: 1, panelId: 5, serialNumber: 123456}
7. Parse control message Route to panel_handler()
8. FFI: read_panel_data() (ip: "192.168.1.100", panel: 5)
9. BACnet Who-Is broadcast
10. I-Am response


11. Read-Property-Multiple
AI 0-49 (all analog inputs)
12. Response: 50 input values
13. Read-Property-Multiple
AO 0-31 (all analog outputs)
14. Response: 32 output values
15. Read-Property-Multiple
AV 0-99 (all variables)
16. Response: 100 variable vals
17. Return DeviceData struct
18. BEGIN TRANSACTION
19. DELETE FROM inputs WHERE panel_id=5
20. INSERT INTO inputs (50 rows)
21. DELETE FROM outputs WHERE panel_id=5
22. INSERT INTO outputs (32 rows)



API Endpoints Called

1. GET /api/t3_device/panels

HTTP

```
GET http://localhost:9103/api/t3_device/panels HTTP/1.1
Host: localhost:9103
Accept: application/json

Response 200 OK:
[
  {
    "id": 5,
    "serial_number": 123456,
    "product_name": "T3-BB",
    "ip_address": "192.168.1.100",
    "port": 47808,
    "location": "Building A - Floor 2",
    "modbus_id": 1,
    "panel_number": 5,
    "online": true
  }
]
```

2. WebSocket Control Message: SELECT_PANEL

JSON

```
{
  "action": 1,
  "source": 1,
  "panelId": 5,
  "serialNumber": 123456,
  "entryType": 0,
  "data": null
}
```

3. WebSocket Response: UPDATE_WEBVIEW_LIST

JSON


```
{
  "action": 16,
  "source": 0,
  "panelId": 5,
  "serialNumber": 123456,
  "entryType": 1,
  "data": {
    "inputs": [
      {
        "number": 0,
        "label": "Room Temperature",
        "value": 72.5,
        "units": "°F",
        "autoManual": "AUTO",
        "rangeLow": 32.0,
        "rangeHigh": 120.0,
        "objectinstance": 3000000
      },
      // ... 49 more inputs
    ],
    "outputs": [
      {
        "number": 0,
        "label": "Cooling Valve",
        "value": 65.0,
        "autoManual": "AUTO",
        "rangeLow": 0.0,
        "rangeHigh": 100.0,
        "objectinstance": 3000100
      },
      // ... 31 more outputs
    ],
    "variables": [
      {
        "number": 0,
        "label": "Cooling Setpoint",
        "value": 75.0,
        "units": "°F",
        "autoManual": "AUTO",
        "objectinstance": 3000200
```

```
      },
      // ... 99 more variables
    ]
  }
}
```

React Component Flow

Step 1: DeviceTree Component

TYPESCRIPT

```typescript
// File: src/t3-react/components/DeviceTree/DeviceTree.tsx
const DeviceTree: React.FC = () => {
  const [panels, setPanels] = useState<Panel[]>([]);

  useEffect(() => {
    // Load panels on mount
    fetch('http://localhost:9103/api/t3_device/panels')
      .then(res => res.json())
      .then(data => setPanels(data));
  }, []);

  const handleDeviceClick = (panel: Panel) => {
    // Dispatch action to load panel
    deviceStore.selectPanel(panel.id, panel.serial_number);
  };

  return (
    <Tree>
      {panels.map(panel => (
        <TreeItem
          key={panel.id}
          onClick={() => handleDeviceClick(panel)}
        >
          {panel.product_name} - {panel.location}
        </TreeItem>
      ))}
    </Tree>
  );
};
```

Step 2: Device Store (Zustand)

TYPESCRIPT

```typescript
// File: src/t3-react/stores/deviceStore.ts
interface DeviceState {
  currentPanelId: number | null;
  inputs: Input[];
  outputs: Output[];
  variables: Variable[];
  isLoading: boolean;
  selectPanel: (panelId: number, serialNumber: number) => void;
  updateData: (data: DeviceData) => void;
}

export const useDeviceStore = create<DeviceState>((set) => ({
  currentPanelId: null,
  inputs: [],
  outputs: [],
  variables: [],
  isLoading: false,

  selectPanel: (panelId, serialNumber) => {
    set({ isLoading: true, currentPanelId: panelId });

    // Send WebSocket message
    websocket.send(JSON.stringify({
      action: 1,  // SELECT_PANEL
      source: 1,  // WebUI
      panelId,
      serialNumber
    }));
  },

  updateData: (data) => {
    set({
      inputs: data.inputs,
      outputs: data.outputs,
      variables: data.variables,
      isLoading: false
    });
  }
}));
```

Step 3: WebSocket Hook

TYPESCRIPT

```typescript
// File: src/t3-react/hooks/useWebSocket.ts
export const useWebSocket = () => {
  const deviceStore = useDeviceStore();
  const [ws, setWs] = useState<WebSocket | null>(null);

  useEffect(() => {
    const websocket = new WebSocket('ws://localhost:9103/ws');

    websocket.onmessage = (event) => {
      const message = JSON.parse(event.data);

      if (message.action === 16) {  // UPDATE_WEBVIEW_LIST
        deviceStore.updateData(message.data);
      }
    };

    setWs(websocket);

    return () => websocket.close();
  }, []);

  return ws;
};
```

Step 4: Data Display Components

TYPESCRIPT

```tsx
// File: src/t3-react/components/InputsTable/InputsTable.tsx
const InputsTable: React.FC = () => {
  const { inputs, isLoading } = useDeviceStore();

  if (isLoading) {
    return <Spinner label="Loading inputs..." />;
  }

  return (
    <Table>
      <TableHeader>
        <TableRow>
          <TableHeaderCell>Number</TableHeaderCell>
          <TableHeaderCell>Label</TableHeaderCell>
          <TableHeaderCell>Value</TableHeaderCell>
          <TableHeaderCell>Units</TableHeaderCell>
          <TableHeaderCell>Mode</TableHeaderCell>
        </TableRow>
      </TableHeader>
      <TableBody>
        {inputs.map(input => (
          <TableRow key={input.objectinstance}>
            <TableCell>{input.number}</TableCell>
            <TableCell>{input.label}</TableCell>
            <TableCell>{input.value}</TableCell>
            <TableCell>{input.units}</TableCell>
            <TableCell>{input.autoManual}</TableCell>
          </TableRow>
        ))}
      </TableBody>
    </Table>
  );
};
```

Backend Handler Implementation

Rust Panel Handler

RUST

```rust
// File: api/src/t3_device/panel_handler.rs
use crate::ffi_interface::read_panel_data;
use crate::db_connection::Database;

pub async fn handle_select_panel(
    msg: ControlMessage,
    db: Arc<Mutex<Database>>,
    clients: Arc<Mutex<Vec<WebSocketClient>>>
) -> Result<(), Error> {
    // Step 1: Get panel info from database
    let panel = {
        let db_lock = db.lock().unwrap();
        db_lock.query_row(
            "SELECT ip_address, port, serial_number FROM panels WHERE id = ?",
            params![msg.panel_id],
            |row| {
                Ok(Panel {
                    ip: row.get(0)?,
                    port: row.get(1)?,
                    serial: row.get(2)?
                })
            }
        )?
    };

    // Step 2: Call FFI to read device via BACnet
    let device_data = unsafe {
        read_panel_data(
            CString::new(panel.ip)?.as_ptr(),
            panel.port,
            msg.panel_id
        )
    };

    if device_data.is_null() {
        return Err(Error::DeviceReadFailed);
    }

    // Step 3: Convert C++ data to Rust structs
    let data = unsafe {
```

```
        DeviceData::from_ffi(device_data)
    };

    // Step 4: Update database cache
    {
        let db_lock = db.lock().unwrap();
        let tx = db_lock.transaction()?;

        // Clear old data
        tx.execute("DELETE FROM inputs WHERE panel_id = ?", params![msg.panel_id])?;
        tx.execute("DELETE FROM outputs WHERE panel_id = ?", params![msg.panel_id])?;
        tx.execute("DELETE FROM variables WHERE panel_id = ?", params![msg.panel_id])?;

        // Insert new data
        for input in &data.inputs {
            tx.execute(
                "INSERT INTO inputs (panel_id, number, label, value, units, auto_manual,
                 VALUES (?, ?, ?, ?, ?, ?, ?)",
                params![
                    msg.panel_id, input.number, input.label,
                    input.value, input.units, input.auto_manual, input.objectinstance
                ]
            )?;
        }

        // Similar for outputs and variables...

        tx.commit()?;
    }

    // Step 5: Broadcast to all WebSocket clients
    let response = ControlMessage {
        action: 16,  // UPDATE_WEBVIEW_LIST
        source: 0,   // Server
        panel_id: msg.panel_id,
        serial_number: panel.serial,
        entry_type: 1,
        data: Some(serde_json::to_value(data)?),
    };

    let clients_lock = clients.lock().unwrap();
```

```
    for client in clients_lock.iter() {
        client.send(serde_json::to_string(&response)?).await?;
    }

    Ok(())
}
```

FFI Interface

RUST

```
// File: api/src/ffi_interface.rs
#[repr(C)]
pub struct CDeviceData {
    pub inputs: *mut CInput,
    pub inputs_count: usize,
    pub outputs: *mut COutput,
    pub outputs_count: usize,
    pub variables: *mut CVariable,
    pub variables_count: usize,
}

#[link(name = "bacnet_ffi")]
extern "C" {
    pub fn read_panel_data(
        ip: *const c_char,
        port: u16,
        panel_id: i32
    ) -> *mut CDeviceData;

    pub fn free_device_data(data: *mut CDeviceData);
}
```

Performance Metrics

Network Timeline

TEXT

```
T+0ms:    User clicks device
T+10ms:   GET /api/t3_device/panels completes
T+15ms:   WebSocket SELECT_PANEL sent
T+20ms:   Backend receives message
T+25ms:   FFI call initiated
T+100ms:  BACnet Who-Is broadcast
T+150ms:  Device I-Am received
T+200ms:  Read AI 0-49 started
T+800ms:  AI values received (600ms for 50 reads)
T+850ms:  Read AO 0-31 started
T+1300ms: AO values received (450ms for 32 reads)
T+1350ms: Read AV 0-99 started
T+2550ms: AV values received (1200ms for 100 reads)
T+2600ms: Database transaction started
T+2750ms: All inserts completed (150ms)
T+2800ms: WebSocket UPDATE_WEBVIEW_LIST sent
T+2850ms: React state updated
T+2900ms: UI re-rendered

Total: ~2.9 seconds
```

Data Sizes

- BACnet request per object: 48 bytes
- BACnet response per object: ~80 bytes
- Total network traffic: ~23 KB
- WebSocket JSON payload: ~150 KB
- Database transaction: ~182 inserts

Memory Usage

- C++ DeviceData struct: ~50 KB
- Rust deserialized data: ~80 KB
- React state: ~150 KB
- DOM rendering: ~200 KB
- Total: ~480 KB per device

Error Scenarios

Device Offline

TEXT

```
User clicks → GET /panels (✓) → WS message (✓) → FFI call (✗ timeout 5s)
                                                          ↓
                                              Error message: "Device not responding"
                                                          ↓
                                              UI shows: Red badge on device tree item
```

Network Timeout

TEXT

```
Reading AI 0-49 (✓) → Reading AO 0-31 (✗ timeout after 30/32)
                                          ↓
                              Partial data saved to database
                                          ↓
                      UI shows: Inputs ✓, Outputs (30/32) ⚠, retry button
```

Database Lock

TEXT

```
User A loading Panel 5 → Transaction open → User B clicks Panel 6
                                                    ↓
                                    Wait for lock (max 2s) → Timeout
                                                    ↓
                                    Show "Please wait" → Retry automatically
```

Optimization Techniques

Batch Reading

RUST

```rust
// Instead of 50 separate BACnet calls:
for i in 0..50 {
    read_property(AI, i);  // 50 round-trips = slow
}

// Use Read-Property-Multiple:
read_property_multiple(AI, vec![0..50]);  // 1 round-trip = fast
```

Parallel Processing

RUST

```rust
// Read inputs, outputs, variables in parallel
let (inputs, outputs, variables) = tokio::join!(
    async { ffi::read_inputs(panel_id) },
    async { ffi::read_outputs(panel_id) },
    async { ffi::read_variables(panel_id) }
);
```

Progressive Loading

TYPESCRIPT

```typescript
// Load critical data first
deviceStore.selectPanel(panelId, { priority: 'inputs,outputs' });
// UI shows inputs/outputs immediately

// Load rest in background
setTimeout(() => {
  deviceStore.loadAdditionalData(panelId, { types: 'variables,programs' });
}, 1000);
```

Database Schema Details

SQL

```sql
-- Inputs table
CREATE TABLE inputs (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  panel_id INTEGER NOT NULL,
  number INTEGER NOT NULL,
  label TEXT,
  value REAL,
  units TEXT,
  auto_manual TEXT DEFAULT 'AUTO',
  range_low REAL,
  range_high REAL,
  filter INTEGER DEFAULT 3,
  objectinstance INTEGER NOT NULL,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  UNIQUE(panel_id, objectinstance),
  FOREIGN KEY (panel_id) REFERENCES panels(id)
);

CREATE INDEX idx_inputs_panel ON inputs(panel_id);
CREATE INDEX idx_inputs_objectinstance ON inputs(objectinstance);
```

File Locations Reference

- **Frontend**: `src/t3-react/components/DeviceTree/`
- **Store**: `src/t3-react/stores/deviceStore.ts`
- **WebSocket**: `src/t3-react/hooks/useWebSocket.ts`
- **Backend Handler**: `api/src/t3_device/panel_handler.rs`
- **FFI Interface**: `api/src/ffi_interface.rs`
- **Database**: `api/src/db_connection.rs`
- **C++ BACnet**: `api/build/bacnet_ffi.dll`